Student guide | Lesson 5 of 5

Score, crashing and reset

Add a score and a clean restart to finish the game.

Name: _____ Date: _____

Today you will make

A score on screen, and a clean restart when the snake hits a wall or its own tail. At the end of this lesson your game is finished.

Words you will meet

Word	What it means
global	Lets a function change a variable that lives outside it.
any()	True if at least one item in a group passes a test.
f-string	Text with variables inside {braces}, like f"Score: {score}".
reset	Put everything back to how it was at the start.
DRY	Don't Repeat Yourself. If you wrote the same code twice, make it a function.

Before you start

- Open snake4.py and save it as **snake5.py**.

Step 1: Score and scoreboard

Add this **after the segments list**:

```
score = 0

scoreboard = turtle.Turtle()
scoreboard.color("white")
scoreboard.penup()
scoreboard.hideturtle()
scoreboard.goto(0, 260)
```

What it does

- **score** counts how many pieces of food we have eaten.
- The scoreboard is a turtle that does not move. It only writes text.
- **hideturtle()** hides the arrow, so only the writing shows.

Step 2: Show the score

Add this **above game_loop**:

```
def update_score():
    scoreboard.clear()
    scoreboard.write(f"Score: {score}", align="center",
                     font=("Arial", 18, "normal"))
```

What it does

- **clear()** wipes the old text so numbers do not pile up.

- **write(...)** prints the new score. The f before the quote lets us put {score} inside the text.

Step 3: Reset the game

Add this **under update_score**:

```
def reset():
    global score
    head.goto(0, 0)
    head.direction = "stop"
    for seg in segments:
        seg.hideturtle()
    segments.clear()
    score = 0
    update_score()
```

What it does

- **global score** tells Python we mean the real score, not a new one made inside the function.
- Hiding every segment before clearing the list removes the old tail from the screen.
- This one function will handle both kinds of crash.

Step 4: The finished game loop

Replace your whole game_loop with this version:

```
def game_loop():
    global score

    if abs(head.xcor()) > LIMIT or abs(head.ycor()) > LIMIT:
        reset()

    if head.distance(food) < STEP:
        move_food()
        add_segment()
        score += 1
        update_score()

    move_body()
    move_head()

    if any(seg.distance(head) < STEP for seg in segments):
        reset()

    screen.update()
    screen.ontimer(game_loop, 100)
```

What it does

- Hitting the wall now calls reset().
- **score += 1** adds one to the score each time you eat.
- The last **if** reads like English: if any segment is touching the head, reset.

Step 5: Show the score from the start

Add one line **just before game_loop()** near the bottom, so the end of your file looks like this:

```
update_score()
game_loop()
screen.mainloop()
```

What it does

- Without this, the score only appears after you eat the first food.

Try it: Run it. Eat food, watch the score climb, then crash into a wall and into your own tail.

Think about it: Where would the same reset code have been written twice if we had not made reset()?

Your answer: _____

Think about it: What happens if you delete global score from reset()? Test it.

Your answer: _____

Challenges (pick one, or do them all)

[ ] Speed the game up a little every time the snake eats. Hint: change the 100 in ontimer.

[ ] Add a high score that stays after a reset.

[ ] Add a second food worth 5 points that disappears after a few seconds.

Stuck? Check here first

What you see	Likely cause and fix
Score stays at 0 or goes wrong	global score is missing in game_loop or reset.
Score text overlaps itself	scoreboard.clear() is missing from update_score().
Old tail pieces stay on screen after a crash	The seg.hideturtle() loop is missing from reset().
The snake resets as soon as it eats	move_body() must run before move_head().

I can...

[ ] show text on screen and update it.

[ ] use global to change a variable from inside a function.

[ ] replace repeated code with one function.

[ ] build a complete game!

Check your work: the complete code

Compare your file with this one, line by line. If yours does not run, the mistake is usually a typo, a missing colon, or indentation that is not lined up.

```
import turtle
import random

STEP = 20
LIMIT = 280

screen = turtle.Screen()
screen.title("Snake")
screen.bgcolor("black")
screen.setup(600, 600)
screen.tracer(0)

head = turtle.Turtle("square")
head.color("lime")
head.penup()
head.direction = "stop"

food = turtle.Turtle("circle")
food.color("red")
food.penup()
food.goto(0, 100)

segments = []
score = 0

scoreboard = turtle.Turtle()
scoreboard.color("white")
scoreboard.penup()
scoreboard.hideturtle()
scoreboard.goto(0, 260)

def go_up():
    if head.direction != "down": head.direction = "up"
def go_down():
    if head.direction != "up": head.direction = "down"
def go_left():
    if head.direction != "right": head.direction = "left"
def go_right():
    if head.direction != "left": head.direction = "right"

def move_head():
    if head.direction == "up":    head.sety(head.ycor() + STEP)
    if head.direction == "down":  head.sety(head.ycor() - STEP)
    if head.direction == "left":  head.setx(head.xcor() - STEP)
    if head.direction == "right": head.setx(head.xcor() + STEP)

def move_body():
    for i in range(len(segments) - 1, 0, -1):
        segments[i].goto(segments[i - 1].pos())
    if segments:
        segments[0].goto(head.pos())

def add_segment():
    seg = turtle.Turtle("square")
    seg.color("green")
    seg.penup()
    segments.append(seg)

def move_food():
    food.goto(random.randrange(-LIMIT, LIMIT, STEP),
              random.randrange(-LIMIT, LIMIT, STEP))
```

```
def update_score():
    scoreboard.clear()
    scoreboard.write(f"Score: {score}", align="center",
                     font=("Arial", 18, "normal"))

def reset():
    global score
    head.goto(0, 0)
    head.direction = "stop"
    for seg in segments:
        seg.hideturtle()
    segments.clear()
    score = 0
    update_score()

def game_loop():
    global score

    if abs(head.xcor()) > LIMIT or abs(head.ycor()) > LIMIT:
        reset()

    if head.distance(food) < STEP:
        move_food()
        add_segment()
        score += 1
        update_score()

    move_body()
    move_head()

    if any(seg.distance(head) < STEP for seg in segments):
        reset()

    screen.update()
    screen.ontimer(game_loop, 100)

screen.listen()
screen.onkeypress(go_up, "Up")
screen.onkeypress(go_down, "Down")
screen.onkeypress(go_left, "Left")
screen.onkeypress(go_right, "Right")

update_score()
game_loop()
screen.mainloop()
```